TEMA 4 – Estrategias avanzadas de búsqueda

Introducción

Métodos de búsqueda local y problemas de optimización

- Búsqueda por ascensión de colinas
- Búsqueda por Temple Simulado
- Búsqueda por haz local
- Búsqueda por Algoritmos genéticos
  - Codificación
  - Operadores genéticos
  - Algoritmo Genético

Búsqueda entre adversarios

- Juegos. Decisiones óptimas en juegos
  - Estrategia óptima. Minimax
  - Decisiones óptimas en juegos multi-jugador
  - Minimax-alfa-beta (Poda alfa-beta)
- Decisiones en tiempo real imperfectas. Heurísticas
  - Funciones de evaluación
  - Corte de la búsqueda
- Juegos que incluyen un elemento de posibilidad

INTRODUCCIÓN

Los métodos de búsqueda visto hasta ahora se diseñan para explorar sistemáticamente espacios de búsqueda.

- Esta forma sistemática consiste en mantener uno o más caminos en memoria, y registrar qué alternativas se han explorado en cada punto a lo largo del camino y cuáles no.
- Cuando se encuentra un objetivo, el camino a ese objetivo también constituye una solución al problema.

En muchos problemas, el camino al objetivo es irrelevante.

- Ejemplo: en las 8-reinas lo importante es la configuración final de reinas, no el orden en que se añaden las reinas.
- Esta propiedad se satisface también en aplicaciones reales: diseño de circuitos integrados, optimización de redes, ….

Introducción

Además, en los procesos de búsqueda vistos hasta ahora, todas las decisiones tomadas pueden ser **recuperables**. Al mantener varios caminos hacia la solución, en cualquier momento podemos decidir cual cogemos, pudiendo volver a continuación a una situación descartada anteriormente. Esto nos indica que el problema lo está resolviendo un único agente resolvedor de problemas.

Pero, hay muchos problemas **no recuperables**. Una decisión de un agente en un momento dado tiene que considerar posibles acciones que pueden realizar otros agentes, si ellos intervienen en la resolución del problema. La imprevisibilidad de estos otros agentes pueden introducir muchas posibles contingencias en el proceso de resolución de problemas (entornos multiagente cooperativos y competitivos). Los entornas competitivos, en los cuales los objetivos del agente están en conflicto, dan ocasión a problemas de búsqueda entre adversarios - a menudo conocidos como juegos.

Según este marco, este tema se desarrolla como sigue:

- Inicialmente discutimos distintos métodos de búsqueda local como son la búsqueda mediante Temple Simulado o Algoritmos Genéticos,
- y acabamos presentando la búsqueda entre adversarios.

MÉTODOS DE BÚSQUEDA LOCAL Y PROBLEMAS DE OPTIMIZACIÓN

Si no importa el camino al objetivo, podemos considerar una clase diferente de métodos: los **métodos de búsqueda local**.

- Operan con un único estado actual en lugar de múltiples caminos
- Generalmente se mueve solo a los vecinos del estado
- Los caminos seguidos no suelen ser recordados

Los métodos de búsqueda local tienen dos ventajas claves:

(1) usan muy poca memoria - por lo general una cantidad constante; y

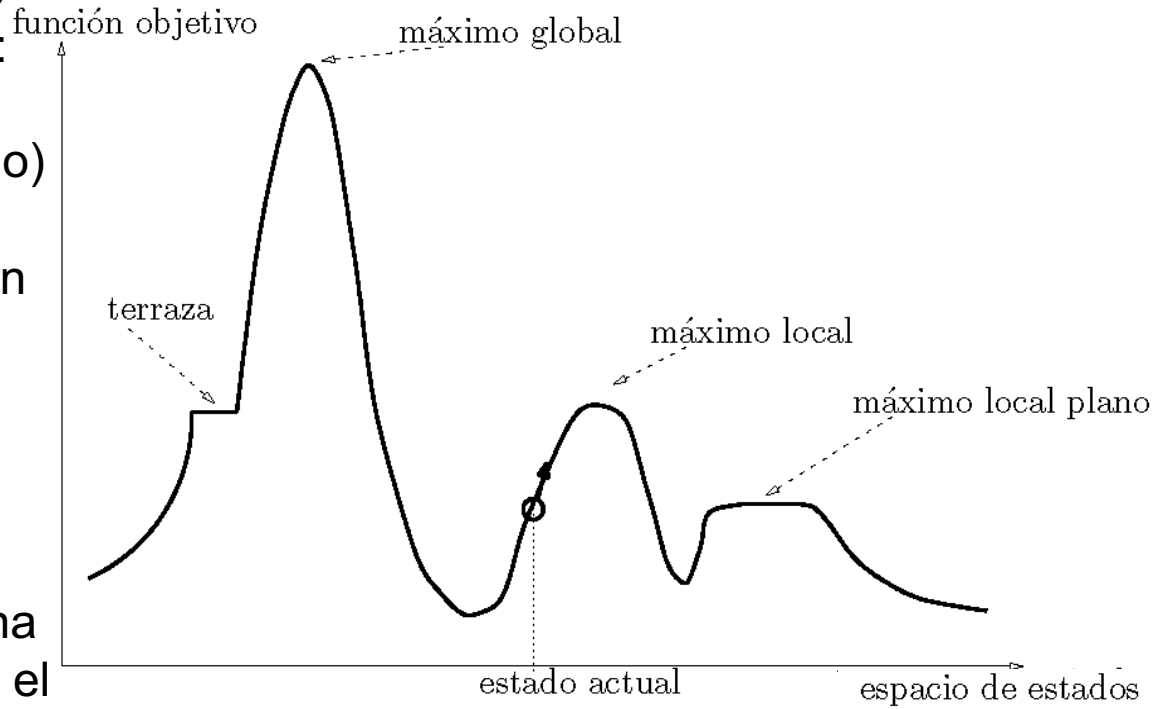
(2) a menudo, pueden encontrar soluciones razonables en espacios de estados grandes o infinitos (continuos) donde los algoritmos sistemáticos son inadecuados.

Los métodos de búsqueda local son útiles para resolver problemas de optimización puros: encontrar el mejor estado según una función objetivo.

Métodos de búsqueda local y problemas de optimización

Es útil considerar que el espacio de estados conforma un paisaje:
El paisaje tiene:

- "posición" (definido por estado)
- "elevación" (definido por la función de evaluación o función objetivo).



Si la elevación corresponde al costo, entonces el objetivo es encontrar el valle más bajo - un mínimo global;
Si la elevación corresponde a una función de evaluación, entonces el objetivo es encontrar el pico más alto - un máximo global.

Los métodos de búsqueda local exploran este paisaje.

Un métodos de búsqueda local completo siempre encuentra un objetivo si existe; uno método óptimo siempre encuentran un mínimo/máximo global.

BÚSQUEDA POR ASCENSIÓN DE COLINAS

El método de búsqueda de ascensión de colinas es simplemente un bucle que continuamente se mueve en dirección del valor creciente. Termina cuando alcanza "un pico" en donde ningún vecino tiene un valor más alto.

El método no mantiene un árbol de búsqueda, sino una estructura de datos del nodo actual que necesita sólo el registro del estado y su valor de función objetivo. La ascensión de colinas no mira delante más allá de los vecinos inmediatos del estado actual.

función ASCENSIÓN-COLINAS(problema) **devuelve** un estado máximo local

entradas: problema, un problema
variables locales: actual y vecino, nodos
actual ← HACER-NODO(ESTADO-INICIAL[problema])
bucle hacer vecino ← sucesor de actual con valor más alto
si VALOR[vecino] ≤ VALOR[actual] **entonces devolver** ESTADO[actual]
actual ← vecino

(también existe la versión para mínimo local: desigualdad a la inversa)

Métodos de búsqueda local y problemas de optimización

Búsqueda por ascensión de colinas

Problema de las 8-reinas →

Formulación completa (8 reinas sobre el tablero, una por columna).

Función sucesor: devuelve todos los estados posibles generados moviendo una reina a otro cuadrado en la misma columna (8×7=56 sucesores).

Función heurística h: número de pares de reinas distintas que se atacan, directa o indirectamente.

El mínimo global de la función heurística es cero - ocurre sólo en soluciones perfectas.

a)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♕	13	16	13	17
♕	14	**17**	15	♕	14	16	16
17	♕	16	18	15	♕	15	♕
18	14	♕	15	15	14	♕	16
14	14	13	17	12	14	12	18

b)

♕							
				♕			
	♕						
					♕		
		♕					
						♕	
			♕				
							♕

(a) Un estado de 8-reinas con una heurística de estimación de costos h=17 (y valores de todos sus sucesores → mejores sucesores con h=12)

(b) Estado mínimo local (h=1) pero cada sucesor tiene un coste más alto

Métodos de búsqueda local y problemas de optimización

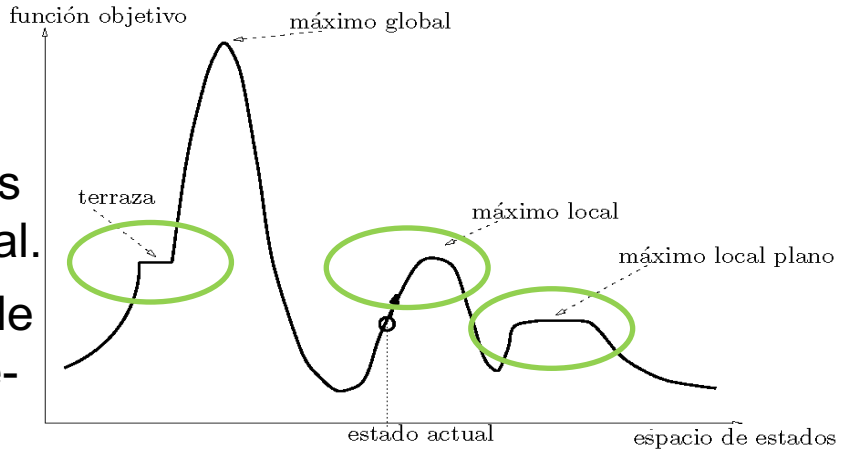
Búsqueda por ascensión de colinas

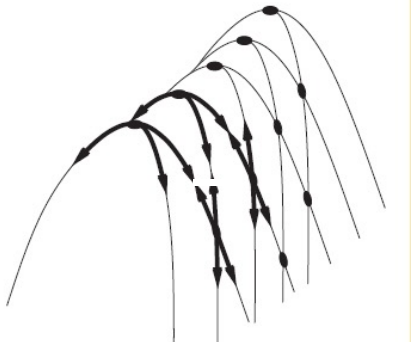
A veces a la ascensión de colinas se le llama búsqueda local avara porque coge un estado vecino bueno sin pensar hacia donde ir después.

En el ejemplo de las 8 reinas, desde el estado inicial se realizan solamente 5 pasos para alcanzar el estado que tiene h=1 y es casi una solución.

La ascensión de colinas a menudo se atasca:

- Máximo local: Un máximo local es un pico que es más alto que cada uno de sus estados vecinos, pero más abajo que el máximo global.
- Crestas: Las crestas causan una secuencia de máximos locales que hace muy difícil la navegación para los métodos avaros.
- Meseta/Terraza: área del paisaje del espacio de estados donde la función objetivo es plana.





En cada caso, el método alcanza un punto en el cual no se puede hacer ningún progreso.

Búsqueda por ascensión de colinas

Comenzando desde un estado de las 8 reinas generado aleatoriamente.

La ascensión de colinas ($8^8 \approx 17$ millones de estados)

→ Se estanca el 86% de veces (resueltos el 14% de los problemas): p=0.14

→ Usa solamente 3 pasos cuando se estanca y 4 pasos cuando tiene éxito

Probabilidad de éxito: p=0.14 (4 pasos)
Probabilidad de fallo: 1-p=0.86 (3 pasos)

El método se para si alcanza una meseta/terraza. ¿Permitir **movimiento lateral**?

Si siempre permitimos movimientos laterales cuando no hay ninguno movimiento ascendente, va a ocurrir un bucle infinito siempre que el método alcance un máximo local plano que no sea una terraza. Una solución común es poner un límite sobre el número de movimientos consecutivos laterales permitidos.

Por ejemplo, si permitimos 100 movimientos laterales consecutivos.

→ Se estanca el 6% de veces (resueltos el 94%): p=0.94

→ 64 pasos cuando se estanca y 21 pasos cuando tiene éxito.

Probabilidad de éxito: p=0.94 (21 pasos)
Probabilidad de fallo: 1-p=0.06 (64 pasos)

Los métodos de ascensión de colinas son incompletos – a menudo dejan de encontrar un objetivo, cuando éste existe (máximos – mínimos locales).

Búsqueda por ascensión de colinas

La **ascensión de colinas de reinicio aleatorio** sería una serie de búsquedas sucesivas desde un estado inicial aleatorio (adopta el refrán "si al principio no tiene éxito, intente otra vez"). Es completa con probabilidad cercana a 1.

Si la búsqueda por ascensión de colinas tiene una probabilidad p de éxito, entonces el número esperado de re-inicios requerido es 1/p.

Por ejemplo, si p=0.2, el número de reinicios (R) necesario para éxito:

$1/R=p \Rightarrow R=1/p=1/0.2=5$ reinicios (4 fallos y 1 acierto)

Los 1/p reinicios se reparten en 1 acierto y (1/p – 1) fallos, y por tanto, el nº total de pasos es el costo de una iteración con éxito más (1-p)/p veces el costo de fallos:

Sin permitir movimientos laterales, $p \approx 0.14$

→ $7 \approx (1/0.14)$ iteraciones para encontrar un objetivo (6 fracasos y 1 éxito).

→ $4+ (0.86/0.14) \times 3=4+(6.14 \times 3)=22.42 \approx 22$ pasos.

Cuando permitimos movimientos laterales, p=0,94

→ $1 \approx (1/0.94)$ iteraciones

→ $(1 \times 21)+ (0.06/0.94) \times 64=21+4.096=25.096 \approx 25$ pasos.

La ascensión de colinas de re-inicio aleatorio es muy eficaz. Incluso para tres millones de reinas, se puede encontrar soluciones en menos de un minuto.

BÚSQUEDA POR TEMPLE SIMULADO

Ascensión de colinas → incompleto y eficaz

Camino puramente aleatorio → completo pero ineficaz

} Necesitamos eficacia y completitud

Búsqueda por Temple Simulado intenta combinar ambas características

La idea se basa en un proceso metalúrgico para templar metal: se calienta a alta temperatura y se enfría gradualmente, facilitando que alcance una estructura cristalina de baja energía

- Metáfora: Bola de ping-pong rebotando en una superficie con oquedades y tratar de colarla en el hueco más profundo:
  - Si se agita con suficiente fuerza puede escapar de mínimos locales
  - Si se agita con demasiada fuerza puede salirse del mínimo global (más profundo)
  - Se disminuye gradualmente la fuerza de agitación conforme el tiempo pasa (equivale a "mapear" el tiempo a la temperatura *T)*

Búsqueda de Temple Simulado

función TEMPLE-SIMULADO(problema, esquema) **devuelve** un estado solución

entradas: problema, un problema

esquema, una aplicación desde el tiempo a ``temperatura''

variables locales: actual y siguiente, nodos

T, una ``temperatura'' controla la probabilidad de un paso hacia abajo

actual ← HACER-NODO(ESTADO-INICIAL[problema])

para t ← 1 to ∞ **hacer**

T ← esquema[t]

si T=0 **entonces devolver** actual

para i ← 1 to L_t **hacer**

siguiente ← un sucesor seleccionado aleatoriamente de actual

ΔE ← VALOR[siguiente] - VALOR[actual]

si $\Delta E>0$ **entonces** actual←siguiente **en caso contrario** actual←siguiente con prob. $e^{\Delta E/T}$

El bucle interno es similar a ascensión de colinas, pero elige sucesor aleatorio

- Si mejora, se acepta siempre
- Si empeora, se acepta con probabilidad $e^{\Delta E/T}$: decrece exponencialmente con la falta de calidad ΔE del sucesor y con la temperatura T (al principio, prob. alta; al final, baja)

BÚSQUEDA POR HAZ LOCAL

El método de búsqueda por haz local guarda la pista de k estados (no solo uno).

- Comienza con estados generados aleatoriamente.
- En cada paso, se generan todos los sucesores de los k estados.
- Si alguno es un objetivo, paramos el método.
- Por otra parte, se seleccionan los k mejores sucesores de la lista completa y repetimos.

A primera vista, podría parecerse a ejecutar k re-inicios aleatorios en paralelo en vez de en secuencia. Pero, los dos métodos son bastantes diferentes. En una búsqueda de reinicio aleatorio, cada proceso de búsqueda se ejecuta independientemente de los demás. En una búsqueda por haz local, la información útil es pasada entre los k hilos paralelos de búsqueda.

En su forma más simple, la búsqueda de haz local → carencia de diversidad entre los k estados (versión cara de la ascensión de colinas).

Una variante llamada búsqueda de haz estocástica → Algoritmos genéticos

BÚSQUEDA POR ALGORITMOS GENÉTICOS

Un algoritmo genético (AG) es una variante de la búsqueda de haz estocástica en el que los estados sucesores se generan combinando a dos estados padres, más que modificar un solo estado.

- ✓ Los AGs comienzan con un conjunto de k estados generados aleatoriamente llamados población.
- ✓ Cada estado (individuo o cromosoma), está representado (codificado) como una cadena sobre un alfabeto finito. Distintas codificaciones hacen que el AG se comporte de forma diferente.
- ✓ Cada elemento de un estado, individuo o cromosoma es denominado gen.
- ✓ El valor heurístico de un estado, individuo o cromosoma es denominado fitness, idoneidad o fenotipo. La función fitness mide la calidad de los estados. ¡Combinando buenos estados se obtienen estados mejores!
- ✓ Se necesita de un conjunto de operadores (denominados operadores genéticos) que permitan generar nuevos individuos.

La codificación define el tamaño del espacio de búsqueda y el tipo de operadores.

Búsqueda por Algoritmos Genéticos

- **Codificación**

El AG codifica la información de cada solución en una cadena que llamamos cromosoma. Existen unos tipos de cadenas que se han utilizado con éxito en el diseño de un AG, aunque podemos diseñar cadenas más acordes o adecuadas a algunos tipos de problemas. Entre las cadenas más utilizadas, nos encontramos con cadenas de 0s y 1s, cadenas de enteros, cadenas de reales, etc)

Codificación binaria

1	0	1	0	0	0	1	1

gen

Por ejemplo, puede representar al número:

$1\times2^7+0\times2^6+1\times2^5+0\times2^4+0\times2^3+0\times2^2+1\times2^1+1\times2^0=163$

ó $n^o\in[2.5,20.5] \rightarrow 2.5+(163/255)\times(20.5-2.5)=14.0059$

Codificación entera

1	2	6	8	2	3	1	4

Entre otros, se utiliza en problemas de localización

Codificación de orden

7	3	6	8	2	4	1	5

Los individuos se representan como permutaciones. Entre otros, se utilizan para problemas de secuenciación.

Codificación real

Una forma natural de codificar una solución es utilizando valores reales como genes. Muchas aplicaciones tienen esta forma natural de codificación.

Búsqueda por Algoritmos Genéticos

- **Operadores genéticos**

El AG dispone de 3 operadores genéticos básicos:

Selección: encargados de escoger qué individuos van a disponer de oportunidades de reproducirse y cuáles no. Habrá individuos que aparezcan más de una vez e individuos que no aparezcan:

Cruce: una vez seleccionados los individuos, éstos son recombinados para producir la descendencia que se insertará en la siguiente generación.

Mutación: la mutación provoca que algunos de los genes de un individuo varíe su valor de forma aleatoria.

A la hora de definir estos operadores genéticos debemos tener en cuenta que ellos son dependientes del tipo de codificación utilizada. Por tanto, existen operadores ya definidos para las codificaciones más utilizadas (codificación binaria, entera, de orden, real, etc). Si diseñamos nuestro tipo particular de codificación, es necesario definir los operadores genéticos adecuados a ellos.

Búsqueda por Algoritmos Genéticos

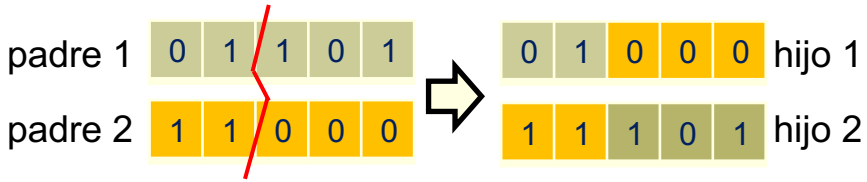
Algunos de los operadores genéticos utilizados son los siguientes:

Selección:

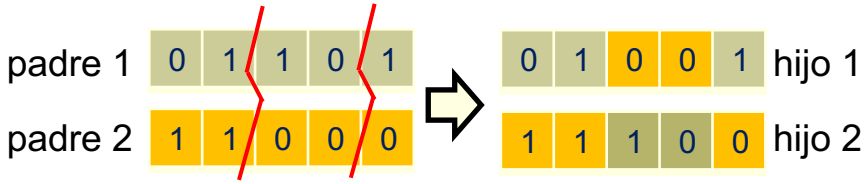
- Ruleta: Se eligen con probabilidad proporcional a su función de idoneidad.
- Torneo: Se establecen k *torneos* aleatorios entre parejas de individuos y se eligen los que ganan en cada torneo (mejor función idoneidad).

Cruce:

- Cruce por un punto.

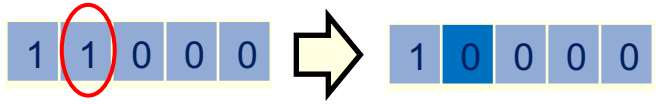


- Cruce por dos puntos.

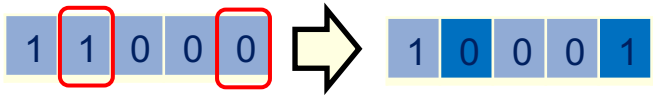


Mutación:

- Cambio de un gen aleatorio.



- Intercambio entre dos genes.



Búsqueda por Algoritmos Genéticos

- **Algoritmo Genético**

Esquemáticamente, el método de un AG es el siguiente:

Se comienza con una población de *k* estados generados aleatoriamente.

Iterar hasta que la población converge o alcanza una condición de parada.

1. Se seleccionan k individuos de la población actual para crear la población intermedia.
2. Se eligen los individuos con una probabilidad (p_c) para ser cruzados.

   Se emparejan, y para cada pareja se aplica el operador de cruce.

   Se obtienen dos nuevos individuos, que sustituyen a los padres.
3. Con una probabilidad (p_m) se mutan los genes de los individuos de la población actual.
4. Esta nueva población obtenida (mediante selección, cruce y mutación) forman la población inicial de la siguiente generación.

Búsqueda por Algoritmos Genéticos



función Algoritmo Genético
  t = 0
  inicializar P(t)
  evaluar P(t)
  mientras (no condición de parada) **hacer**
    t = t + 1
    seleccionar P(t) desde P(t-1)
    cruzar P(t)
    mutación P(t)
    evaluar P(t)

Un algoritmo genético más detallado (con el operador de cruce por un punto y la mutación de un gen), es el siguiente:

Búsqueda por Algoritmos Genéticos

función ALGORITMO-GENÉTICO(población,IDONEIDAD) **devuelve** un individuo

entradas: población; función idoneidad
repetir
nueva_población ← conjunto vacio
nueva_población ← SELECCIÓN(población,IDONEIDAD)
población_cruzar ← conjunto vacio
para x desde 1 **hasta** TAMAÑO(nueva_población) **hacer**
si ($r<p_c$) **entonces** población_cruzar←problación_cruzar+x
para i desde 1 **hasta** TAMAÑO(población_cruzar) **hacer**
x, y←elegir aleatoriamente 2 individuos de población_cruzar
$hijo_1$, $hijo_2$ ← CRUCE(x,y)
sustituir x e y por $hijo_1$ e $hijo_2$ en nueva_población
para x desde 1 **hasta** TAMAÑO(nueva_población)×LONGITUD(individuo) **hacer**
si ($r<p_m$) **entonces** gen←MUTAR(gen) y modificar gen del hijo de nueva_población
población ← nueva_población
hasta que algún individuo es bastante adecuado, o ha pasado bastante tiempo
devolver el mejor individuo en la población, de acuerdo con la IDONEIDAD

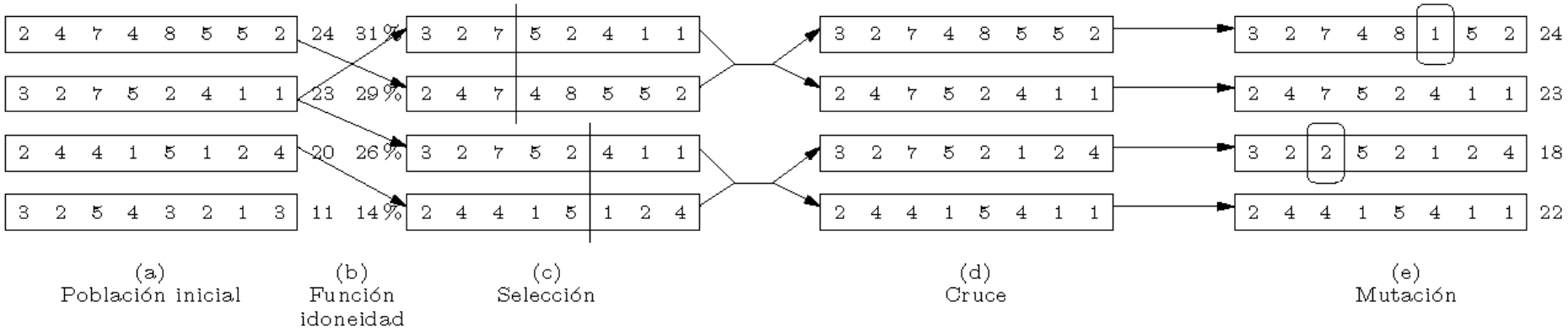
---

función CRUCE(x,y) **devuelve** un individuo
entradas: x,y, padres individuales
n ← LONGITUD(x)
c ← número aleatorio de 1 a n
devolver $individuo_1$ ← CONCATENAR(SUBCADENA(x,1,c),SUBCADENA(y,c+1,n))
$individuo_2$ ← CONCATENAR(SUBCADENA(y,1,c),SUBCADENA(x,c+1,n))

Búsqueda por Algoritmos Genéticos

8 reinas:

→ cadenas de enteros, cada uno en el rango de 1 a 8
(podría ser cadenas de 0s y 1s, donde se requieren $8 \times \log_2 8 = 24$ genes)



(a) Población inicial
(b) Función idoneidad (calculamos el porcentaje de la ruleta de cada individuo)
(c) Población seleccionada mediante el método de la ruleta
(c) De la población, se seleccionan (mediante p_c) los individuos a ser cruzados
(d) Resultando de los cruces
(d) De esta nueva población, se seleccionan (mediante p_m) los genes a ser mutados
(e) Resultados de las mutaciones (p_m) → Nueva población y repetimos ……

BÚSQUEDA ENTRE ADVERSARIOS

JUEGOS. DECISIONES ÓPTIMAS EN JUEGOS

En los entornas multiagente, cualquier agente tiene que considerar las acciones de los otros agentes y como afectan a su propio bienestar. La imprevisibilidad de estos otros agentes pueden introducir muchas posibles contingencias en el proceso de resolución de problemas del agente (entornos multiagente cooperativos y competitivos). Los entornos competitivos, en los cuales los objetivos del agente están en conflicto, dan ocasión a problemas de **búsqueda entre adversarios** - a menudo conocidos como **juegos**.

En Inteligencia Artificial, "los juegos" son, por lo general, una clase especializada - que los teóricos de juegos llaman **juegos de suma cero**, de dos jugadores, por turnos, determinista, de información perfecta. En nuestra terminología, significan entornos deterministas, totalmente observables con dos agentes cuyas acciones deben alternar y en los que los valores utilidad, al final de juego, son siempre iguales y opuestos (por ejemplo, si un jugador gana (+1), el otro jugador necesariamente pierde (-1)).

Búsqueda entre adversarios

Juegos. Decisiones óptimas en juegos

Consideraremos juegos con dos jugadores (MAX y MIN). MAX mueve primero, y luego mueven por turno hasta que el juego se termina. Al final de juego, se conceden puntos al jugador ganador y penalizaciones al perdedor.

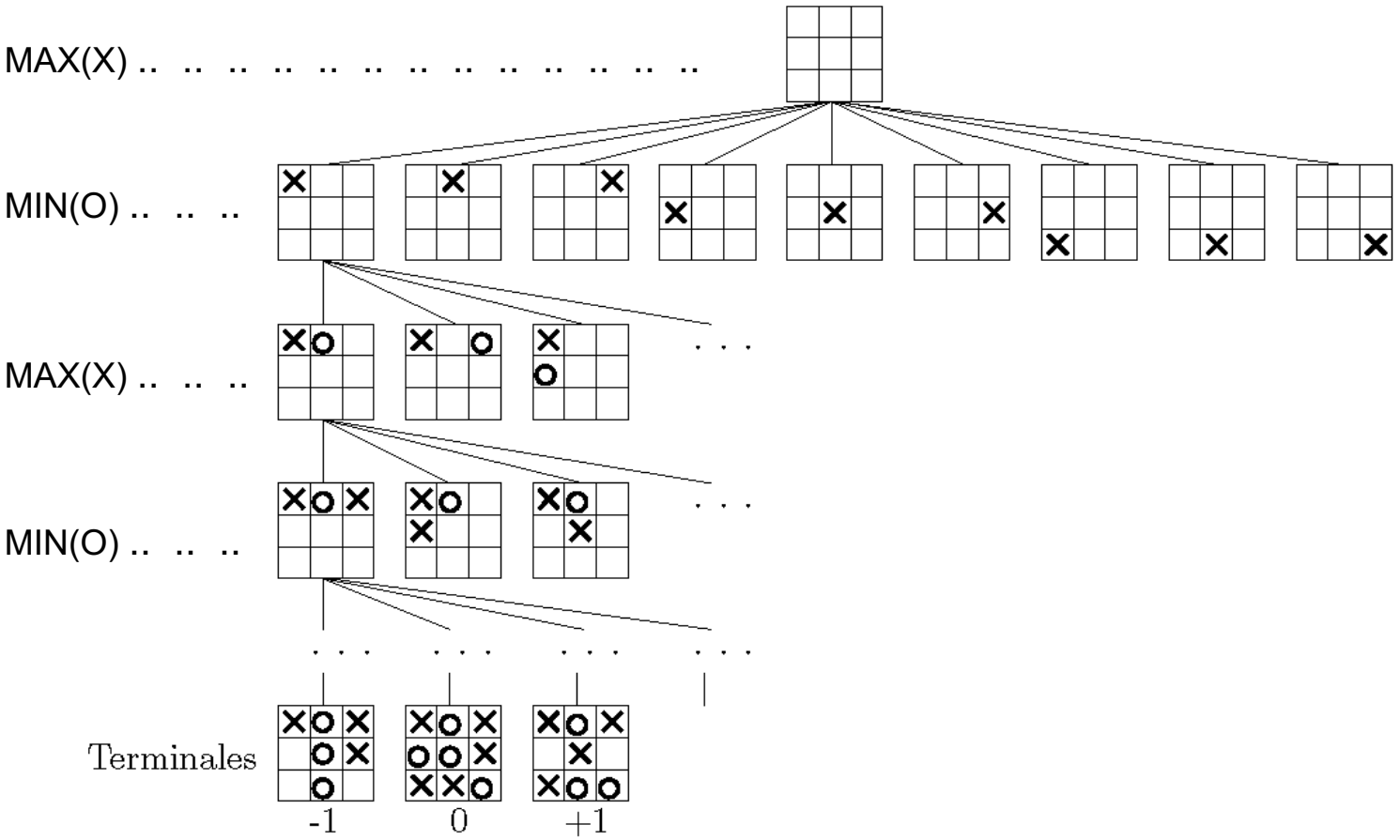
Un "**juego**" puede definirse formalmente como una clase de problemas de búsqueda:

- El **estado inicial**: incluye la posición del tablero e identifica al jugador que mueve.
- Una **función sucesor**: devuelve una lista de pares (movimiento, estado).
- Un **test terminal**: determina cuando se termina el juego (estados terminales).
- Una **función utilidad** (rentabilidad): valor numérico a los estados terminales.

El estado inicial y los movimientos legales para cada lado definen el árbol de juegos. El número sobre cada nodo hoja indica el valor de utilidad del estado terminal desde el punto de vista de un jugador (valores altos, buenos para MAX y malos para MIN). El trabajo de MAX es usar el árbol de búsqueda (en particular la utilidad de estados terminales) para determinar el mejor movimiento.

Búsqueda entre adversarios

Juegos. Decisiones óptimas en juegos



Un árbol (parcial) de búsqueda para el juego de tic-tac-toe (tres en raya)

- **Estrategia óptima. Minimax**

En un problema de búsqueda normal, la solución óptima sería una secuencia de movimientos que conducen a un estado objetivo (estado terminal que es ganador)

Pero en un juego, MIN interfiere. MAX debe encontrar una estrategia contingente, que especifica el movimiento de MAX en el estado inicial, después los movimientos de MAX en los estados que resultan de cada respuesta posible de MIN, después los movimientos de MAX en los estados que resultan de cada respuesta posible de MIN de los anteriores movimientos, etc.

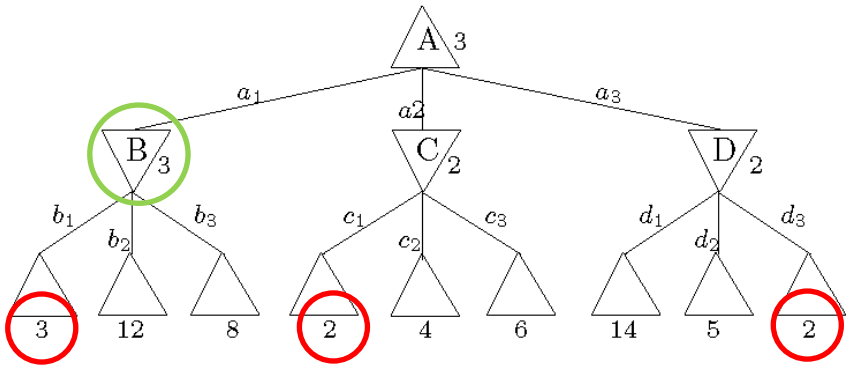
$$\text{VALOR_MINIMAX}(n) = \begin{cases} \text{UTILIDAD}(n) & \text{si n es un nodo terminal} \\ \max_{s \in \text{SUCESORES}(n)} \text{VALOR_MINIMAX}(s) & \text{si n es un nodo MAX} \\ \min_{s \in \text{SUCESORES}(n)} \text{VALOR_MINIMAX}(s) & \text{si n es un nodo MIN} \end{cases}$$

Definición de juego óptimo para MAX suponiendo que MIN juega óptimamente.

¿Y si MIN no juega óptimamente? MAX lo hará aún mejor.

Búsqueda entre adversarios

Juegos. Decisiones óptimas en juegos



función DECISION-MINIMAX(ESTADO) **devuelve** una acción

variables de estrada: estado, estado actual del juego
v ← MAX_VALOR(estado)
devolver la acción de SUCESORES(estado) con valor v

función MAX-VALOR(estado) **devuelve** un valor utilidad

si TEST-TERMINAL(estado) **entonces devolver** UTILIDAD(estado)
v ← -∞
para un s en SUCESORES(estado) **hacer** v ← MAX(v,MIN-VALOR(s))
devolver v

función MIN-VALOR(estado) **devuelve** un valor utilidad

si TEST-TERMINAL(estado) **entonces devolver** UTILIDAD(estado)
v ← ∞
para un s en SUCESORES(estado) **hacer** v ← MIN(v,MAX-VALOR(s))
devolver v

Juegos. Decisiones óptimas en juegos

Método minimax realiza:

exploración primero en profundidad.

Si suponemos que m es la profundidad máxima y b el número de movimientos legales:

→ complejidad en tiempo: $O(b^m)$.

→ complejidad en espacio

- O(bm) si se generan todos los sucesores a la vez,
- O(m) si se generan los sucesores uno por uno

Un árbol MINIMAX es análogo a un árbol Y/O, donde los nodos MAX son nodos O y los nodos MIN son nodos Y (se asume que el oponente es infalible)

Juegos. Decisiones óptimas en juegos

- **Decisiones óptimas en juegos multi-jugador**

Extensión de minimax a juegos multi-jugador.

Sustituir el valor para cada nodo con un vector de valores

(3 jugadores A, B, y C → (v_A, v_B, v_C) asociado con cada nodo).

Para los estados terminales, este vector dará la utilidad del estado desde el punto de vista de cada jugador.

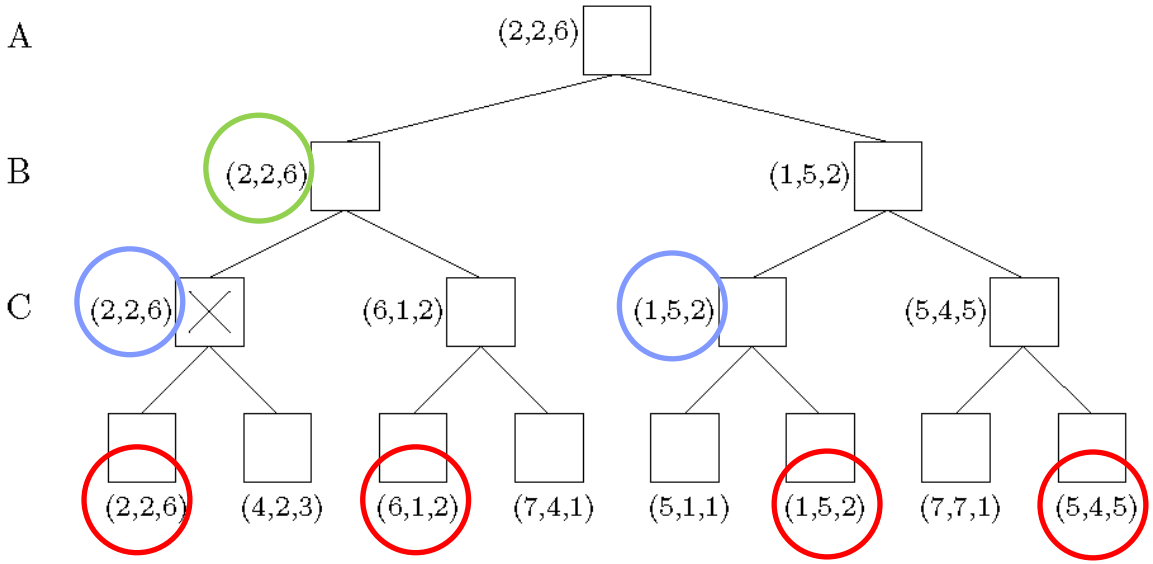
(en juegos de suma cero, el vector de dos elementos puede reducirse a un valor porque los valores son siempre opuestos).

Hacer que la función UTILIDAD devuelva un vector de utilidades.

El valor hacia atrás de un nodo n es el vector de utilidad de cualquier sucesor que tiene el valor más alto para el jugador que elige en n.

Búsqueda entre adversarios

Juegos. Decisiones óptimas en juegos



- **Minimax-alfa-beta (Poda alfa-beta)**

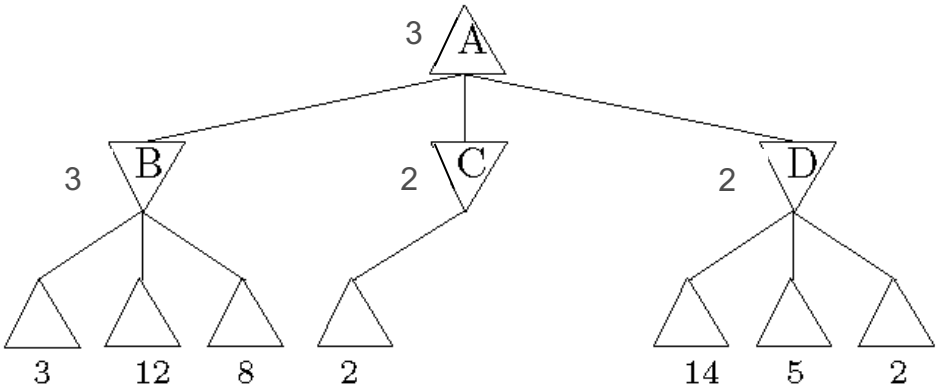
Búsqueda minimax → crecimiento exponencial en el número de movimientos

No se puede eliminar el exponente, pero se puede dividir.

Poda alfa-beta → devuelve el mismo movimiento que devuelve minimax pero podando nodos o subárboles.

α = valor más alto (mejor opción) encontrado a lo largo del camino para MAX

β = valor más bajo (mejor opción) encontrado a lo largo del camino para MIN



Búsqueda entre adversarios

Juegos. Decisiones óptimas en juegos

función BUSQUEDA-ALFA-BETA(ESTADO) **devuelve** una acción

variables de estrada: estado, estado actual del juego
v ← MAX_VALOR(estado,-∞,+∞)
devolver la acción de SUCESORES(estado) con valor v

---

función MAX-VALOR(estado,α,β) **devuelve** un valor utilidad

variables de estrada: estado, estado actual del juego, α, β
si TEST-TERMINAL(estado) **entonces devolver** UTILIDAD(estado)
v← -∞
para un s en SUCESORES(estado) **hacer**
v ← MAX(v,MIN-VALOR(s,α,β))
si v≥β **entonces devolver** v
α ← MAX(α,v)
devolver v

---

función MIN-VALOR(estado,α,β) **devuelve** un valor utilidad

variables de estrada: estado, estado actual del juego, α, β
si TEST-TERMINAL(estado) **entonces devolver** UTILIDAD(estado)
v ← +∞
para un s en SUCESORES(estado) **hacer**
v ← MIN(v,MAX-VALOR(s,α, β))
si v≤α **entonces devolver** v
β ← MIN(β,v)
devolver v

La eficacia de la poda alfa-beta es muy dependiente del orden en el que se examinan los sucesores.

- Si asumimos que esto puede hacerse, resulta que alfa-beta tiene que examinar sólo $O(b^{d/2})$ nodos para escoger el mejor movimiento, en vez de $O(b^{d})$.
- Si los sucesores se examinan en orden aleatorio más que primero el mejor, el número total de nodos examinados será aproximadamente $O(b^{3d/4})$.

Búsqueda entre adversarios

DECISIONES EN TIEMPO REAL IMPERFECTAS. HEURÍSTICAS

Tanto minimax como alfa-beta tiene que buscar en todos los caminos (o una parte) hasta los estados terminales.

Esta profundidad no es, por lo general, práctica porque los movimientos deben hacerse en una cantidad razonable de tiempo - típicamente, unos minutos.

Shannon en 1950 propuso un cambio, que los programas deberían cortar la búsqueda antes y entonces aplicar una función de evaluación heurística a los estados, convirtiendo, efectivamente, los nodos no terminales en hojas terminales.

- Sustituir la función de utilidad por una función de evaluación heurística EVAL que estime la utilidad de la posición, y
- Sustituir el test-terminal por un test-límite que decide cuando aplicar EVAL.

Búsqueda entre adversarios

Decisiones en tiempo real imperfectas. Heurísticas

- **Funciones de evaluación**

Una función de evaluación devuelve una estimación de la utilidad esperada de una posición, tal como hacen las funciones heurísticas que devuelven una estimación de la distancia al objetivo.

- **Corte de la búsqueda**

Tanto en la BÚSQUEDA-MINIMAX y BUSQUEDA-ALFA-BETA se sustituyen las dos líneas que mencionan al TEST-TERMINAL:

si TEST-CORTE(estado, profundidad) entonces devolver EVAL(estado)

Llevar el cálculo de la profundidad y poner límite de profundidad fijo, de modo que TEST-CORTE(estado, profundidad) devuelva verdadero para toda profundidad mayor que alguna profundidad fija d. (también devolverá verdadero para todos los estados terminales). La profundidad d se elige de modo que la cantidad de tiempo usado no exceda de lo que permiten las reglas del juego.

Búsqueda entre adversarios

Decisiones en tiempo real imperfectas. Heurísticas

Test limite más sofisticado.

→ La función de evaluación debería aplicarse sólo a posiciones estables

→ Las no estables se extienden hasta alcanzar posiciones estables.

(búsqueda de estabilidad o de reposo)

Veamos un ejemplo: Tic-Tac-Toe (3 en raya)

- Empieza Max
- Expandimos el árbol de juegos hasta nivel de profundidad 2
- En nivel 2, evaluamos mediante una función heurística:

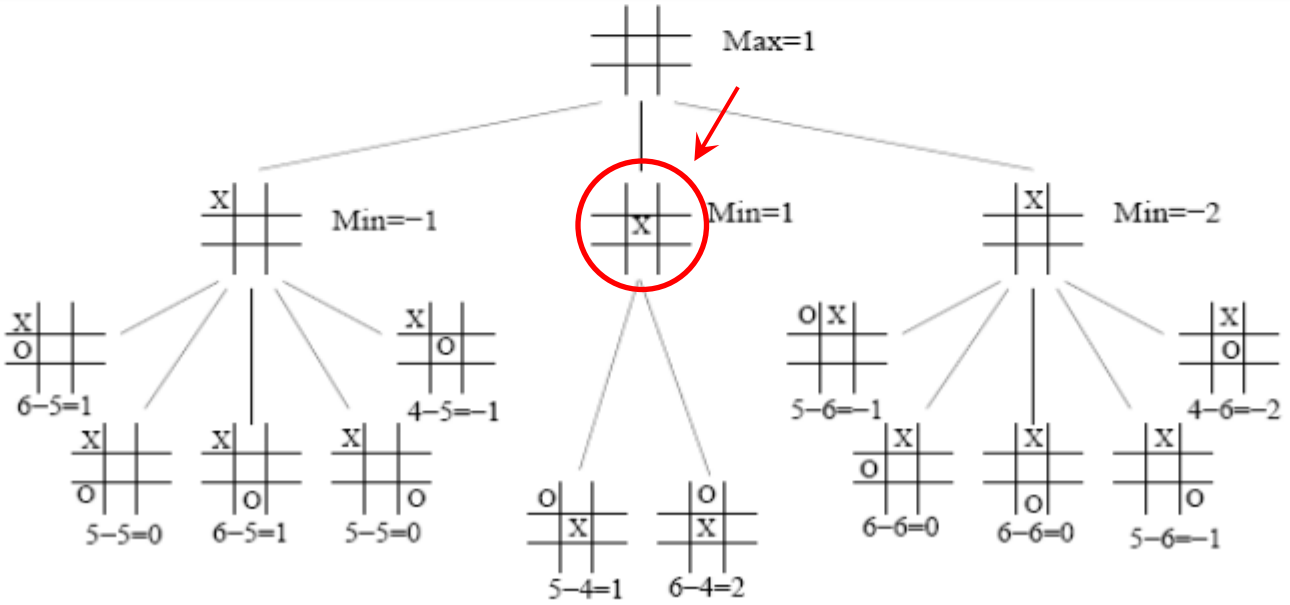
  h(n) = (nº de filas, columnas y diagonales libres para Max) -

  (nº de filas, columnas y diagonales libres para Min)

  Si el nodo que evaluamos es terminal, devuelve $+\infty$ si es solución para Max y $-\infty$ si es solución para Min
- Se ignoran los nodos que, por simetrías, son iguales.

Búsqueda entre adversarios

Decisiones en tiempo real imperfectas. Heurísticas



Como podemos observar, el movimiento que propone minimax al jugador Max (por el que juega) es poner en el centro.

Búsqueda entre adversarios

Decisiones en tiempo real imperfectas. Heurísticas

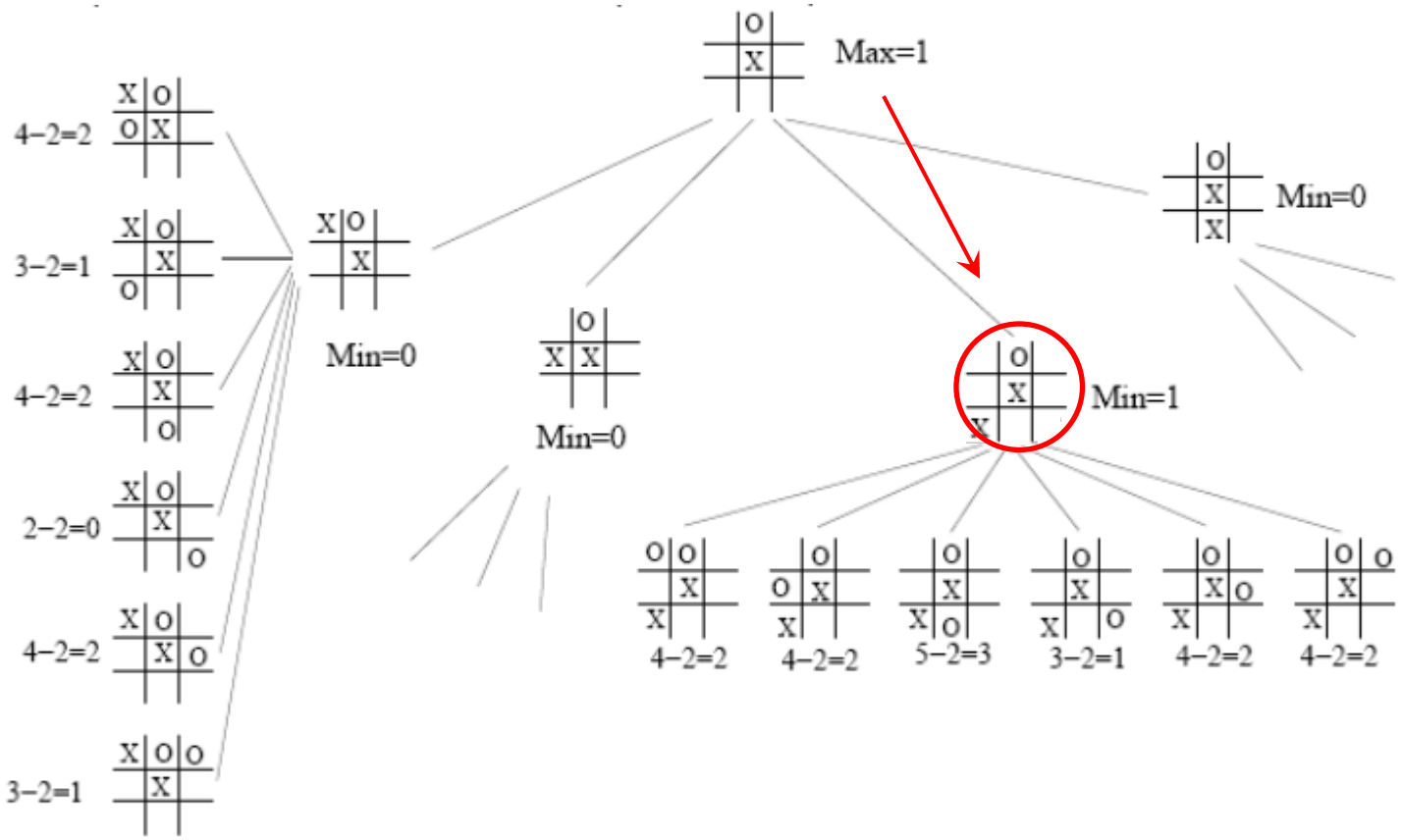
Ante el movimiento de Max, el jugador Min ha realizado su movimiento, generando la situación que se muestra en el nodo raíz de la estructura. A partir de este nodo, minimax construye su posible solución para Max.



Búsqueda entre adversarios

Decisiones en tiempo real imperfectas. Heurísticas

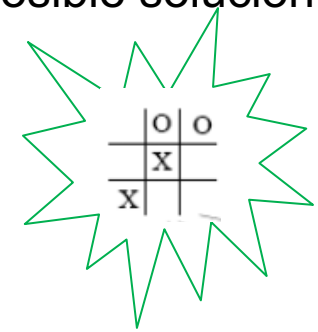
Ante el movimiento de Max, el jugador Min ha realizado su movimiento, generando la situación que se muestra en el nodo raíz de la estructura. A partir de este nodo, minimax construye su posible solución para Max.



Búsqueda entre adversarios

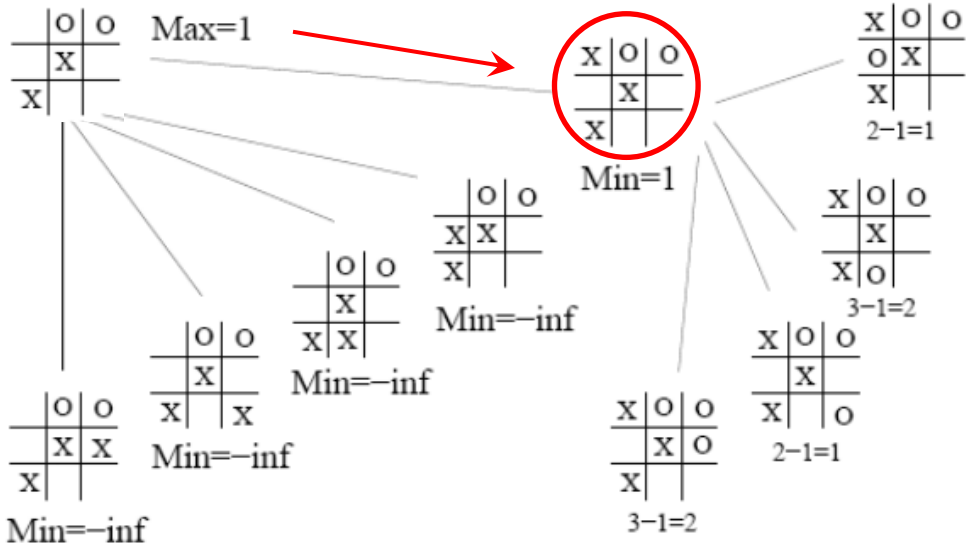
Decisiones en tiempo real imperfectas. Heurísticas

De nuevo, ante el movimiento de Max, el jugador Min ha realizado el suyo, generando la situación que se muestra en el nodo raíz de la estructura. Minimax construye su posible solución para Max.



Decisiones en tiempo real imperfectas. Heurísticas

De nuevo, ante el movimiento de Max, el jugador Min ha realizado el suyo, generando la situación que se muestra en el nodo raíz de la estructura. Minimax construye su posible solución para Max.



Como podemos observar, el movimiento propuesto por minimax produce un nodo que será ganador para Max en los próximos movimientos.

Búsqueda entre adversarios

JUEGOS QUE INCLUYEN EN ELEMENTO DE POSIBILIDAD

Muchos juegos incluyen un elemento aleatorio
(como el lanzamiento de dados)



¿Cómo afecta al proceso de toma de decisiones?

Queremos escoger el movimiento que conduzca a la mejor posición

Como no tienen valores minimax definidos → calcular el valor esperado

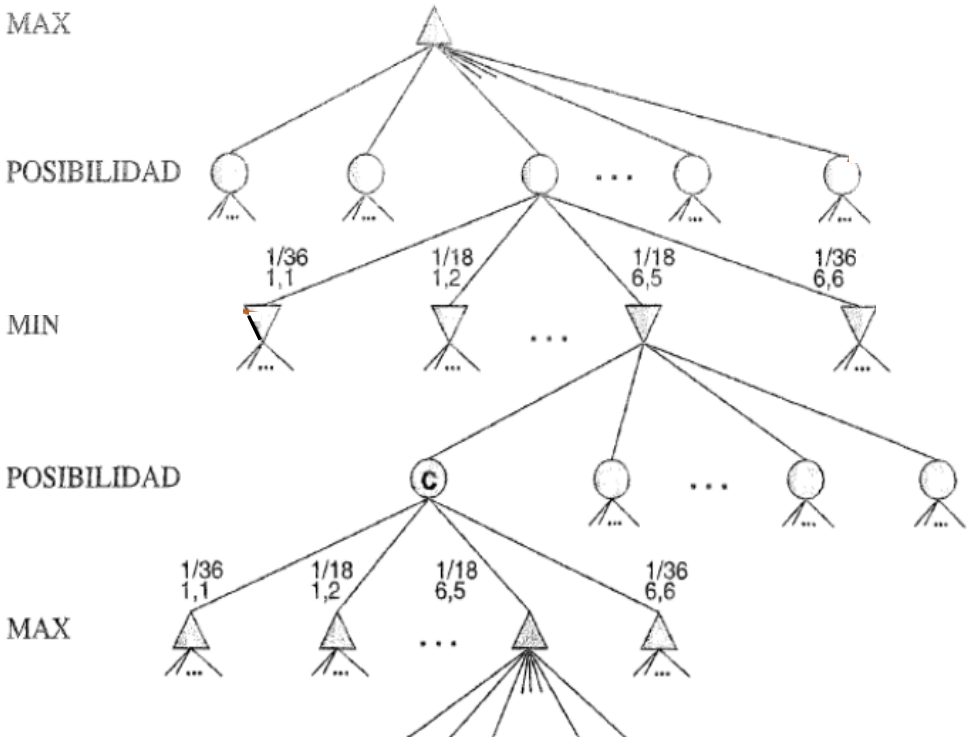
(la expectativa se toma sobre todos los posibles resultados que podrían ocurrir).

Generalizar el valor mínimax para juegos deterministas a un valor minimax esperado para juegos con nodos de posibilidad.

Búsqueda entre adversarios

Juegos que incluyen un elementos de posibilidad

Los nodos terminales y los MAX y MIN trabajan igual que antes; los nodos de posibilidad se evalúan tomando el promedio ponderado de los valores que se obtienen de todos los resultados posibles.



$$\text{MINIMAXESPERADO}(n) = \begin{cases} \text{UTILIDAD}(n) & \text{si } n \text{ es un nodo terminal} \\ \max_{s \in \text{SUCESORES}(n)} \text{MINIMAXESPERADO}(s) & \text{si } n \text{ es un nodo MAX} \\ \min_{s \in \text{SUCESORES}(n)} \text{MINIMAXESPERADO}(s) & \text{si } n \text{ es un nodo MIN} \\ \sum_{s \in \text{SUCESORES}(n)} P(s)\,\text{MINIMAXESPERADO}(s) & \text{si } n \text{ es un nodo posibilidad} \end{cases}$$

donde P(s) es la probabilidad de que ocurra s.